

Nova

Decisions, the Jev access plan, and what's now in the Agentic_OS folder.

Where we are. The open questions from the brainstorm are settled, and a first working version of Nova is in the folder: a local brain (daemon), the glass voice UI, and a model-agnostic decision layer that already speaks Jev's API. It runs today with no keys at all; adding one Vercel key switches System 1 to Jev.

1. Decisions

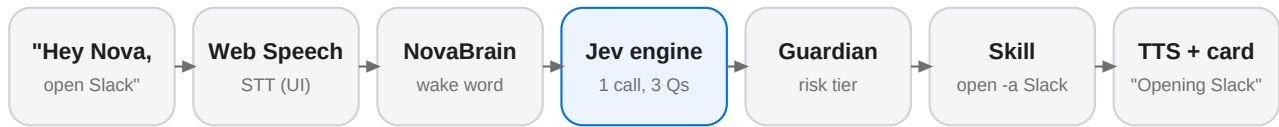
Question	Decision	What it changes
Primary user	You / developers first	Decision Inspector visible by default; Cmd+K typing fallback; terminal-friendly setup.
Listening mode	Always-listening with wake word	Continuous recognition + "Hey Nova" / "Nova"; 8-second follow-up window after each reply.
Jev access	Waitlisted - use Vercel AI Gateway	Model id typesafe-ai/jev via the AI SDK evaluate API; no code change when a direct key arrives.
Name	Nova (placeholder)	Configurable wake words in .env, so a rename is cheap.

2. Getting Jev while on the waitlist

Path	Status	Role in Nova
Vercel AI Gateway (typesafe-ai/jev)	Available now; announced 16 Sep 2026	Primary System 1. Zero-data-retention flag set on every call.
LLM with a schema (Claude, GPT, Gemini...)	Available now, same Vercel key	Fallback + baseline to compare Jev's answers against.
Offline heuristic	Built in	Default when no key is set; also makes tests deterministic.
OpenJev (open-source, local 4B model)	Optional later	Possible offline mode; slower and less accurate than Jev.
Direct TypeSafe key	When the waitlist clears	One more engine slot; nothing else changes.

First real task once you have the Vercel key: `npm run jev:ping`. It times Jev calls from Ghana (the service runs on the US West Coast). That number decides how instant Nova can feel.

3. What's built



jev → fallback on error/timeout

Open questions branch to System 2 (NOVA_BRAIN_MODEL) instead of a skill.

Piece	Where	Notes
DecisionEngine	packages/core/src/decision	Jev / LLM / heuristic behind one interface; hard latency budget with automatic fallback.
NovaBrain	packages/core/src/brain	Wake word, follow-up window, addressee check, slot filling ("Which app?"), confirmations, System 2 hand-off.
Skills	packages/core/src/skills	open app, quit app (needs "yes"), time, timers, cancel. Examples double as Jev criteria.
Guardian	packages/core/src/guardian.ts	Tier set in code: T0 do · T1 announce · T2 spoken confirm · T3 on-screen tap only.
Daemon	apps/daemon	WebSocket on 127.0.0.1 only, rejects other web origins, only opens apps it discovered itself.
Glass UI	apps/desktop/src	Orb, Live Pill, frosted cards (swipe to dismiss), magnifying dock, Inspector, Activity, Cmd+K bar, earcons.
Native shell	apps/desktop/src-tauri	Tauri 2, macOS overlay title bar, mic permission strings.

4. Run it

```
cd ~/dev/Agentic_OS
npm install
cp .env.example .env # optional - add AI_GATEWAY_API_KEY for Jev
npm run dev # daemon + UI
# open http://localhost:5173 in Chrome, tap 'Wake Nova', allow the mic
```

Try: "Hey Nova, open Slack" · "Nova, what time is it" · "Nova, set a timer for 5 minutes" · "Nova, quit Spotify" then "yes" · "Hey Nova", pause, "open Figma". Shortcuts: Cmd+K type, Cmd+I inspector, Cmd+J activity, Esc stop. /#demo plays a scripted session without the daemon.

```
npm run jev:ping # Jev latency from your location
npm test # 17 tests
npm run dev:tauri # native window (needs Rust; run dev:daemon alongside)
```

5. What was verified - and what wasn't

Check	Result
Typecheck (core, daemon, desktop)	Clean
Unit tests - wake word, skills, confirm/decline, slot filling, mocked-Jev thresholds, fallback, System 2 routing, timers	17 / 17 pass
Production UI build (Vite)	Builds
Daemon end to end over WebSocket (offline engine)	Open app, time, timer, quit + "yes", background speech ignored
Other web origins connecting to the daemon	Rejected (401)
Live Jev call	Not run - needs your Vercel key
Visual check of the UI in a browser	Not possible in my sandbox - please look and tell me what to change
Tauri native build	Not run - no Rust toolchain here
Voice recognition inside the macOS Tauri webview	Unknown; use Chrome until verified

6. Next steps

- **You:** create the Vercel AI Gateway key, run `jev:ping`, and share the p50/p90 numbers.
- **Then:** compare Jev vs heuristic vs LLM on ~50 of your real phrases, and pick thresholds from the data.
- **Voice:** move wake word + speech-to-text into the daemon (openWakeWord + on-device Whisper) so it works in the native window and allows barge-in.
- **Agents:** hand off coding tasks to Claude Code / Codex over the Agent Client Protocol; add MCP tools.
- **Safety:** undo for file actions, voice ID, and the untrusted-content firewall before any tool reads the web or email.

Sources: Vercel changelog - TypeSafe AI's Jev now available on AI Gateway (16 Sep 2026); Vercel KB - How to classify, route, and score with Jev and AI SDK; TypeSafe AI - Introducing System One Models & Jev; OpenJev on GitHub.